

Experimento sobre el problema de coloración de grafos — NP-Completo

Thomas Jaramillo Aguirre - Juan Jose Valencia

July 2026

1. Introducción

Los problemas NP-Completo, son un conjunto de problemas los cuales cumplen una serie de condiciones, en primer lugar se deben considerar problemas del tipo NP, problemas de decisión (los cuales su respuesta es afirmativa o negativa), tales que su solución no se puede determinar en un tiempo polinómico, además debe tener la cualidad de que cualquier otro problema de la clase NP puede ser reducido polinomialmente al problema determinado, lo que nos posibilita que al encontrar un algoritmo capaz de dar la solución en tiempo polinómico para al menos uno de los problemas entonces todos podrían ser resueltos en tiempo polinómico.

En esta clase de problemas existe una gran cantidad, los cuales pueden tener instancias extrapolables a la vida real, como lo pueden ser el problema del agente viajero, el problema del morral y el problema del mínimo coloramiento de grafos entre muchos otros, este último problema tiene extrapolabilidad en la programación de exámenes, determinación de conglomerados, coloración de mapas, cuadrados latinos, sudoku y distribución de acientos[1], [2]. Por lo anterior es claro que existe una necesidad en buscar que tipo de algoritmos contribuyen a encontrar soluciones óptimas o aproximadas de las instancias, además de que se puedan determinar en tiempo finito para tener un balance entre aproximación de la solución y su tiempo, ya que hacer la búsqueda de algún algoritmo para la resolución en tiempo polinómico, sería una tarea más demandante que entender el funcionamiento de algoritmos ya conocidos.

2. Problema: Coloración de un Grafo

El problema de coloración de grafos consiste en asignar colores a los vértices de un grafo de manera que los vértices adyacentes (unido por una arista), no compartan el mismo color. Dependiendo de la formulación del problema, el objetivo puede ser determinar si es posible colorear el grafo utilizando un número determinado de colores, problema de decisión, o encontrar la coloración optima

de un grafo, la cual es la coloración mínima para este, que denotaremos como $\chi(G)$, donde G es el grafo de entrada para los algoritmos.

Para el estudio se buscaran algoritmos los cuales resuelvan la coloración mínima del grafo, todos los grafos al ser evaluados deben cumplir una serie de condiciones para ser procesados por cualquier algoritmo, no debe contener lazos ya que estos no permitirían formar la coloración del grafo ya que al evaluar esa arista claramente el grafo tendría el mismo color que el mismo, también deben cumplir que no sean multigrafos, puesto que con el uso de una única arista donde se presente el multigrafo es suficiente para encontrar una solución al problema.

Además se omitiran instancias consideradas triviales o las cuales no contribuyen a un análisis adecuado del problema, en primer lugar todo grafo tal que su conjunto de aristas es vacío, esto genera que ningún vértice tenga vecinos por lo tanto el grafo se puede colorear con tan solo un color; por otro lado los grafos completos representados por k_n , donde n es la cantidad de nodos, los cuales constan en tener una arista para todo par de vértices del grafo, así el mínimo coloramiento posible es exactamente n , y al tener la restricción de no ser multigrafos entonces se logra hacer el cálculo directo para conocer si un grafo es completo antes de iterar cualquier algoritmo, esto se cumple si $A = \frac{n(n-1)}{2}$, donde A es la cardinalidad del conjunto de aristas; por último a los grafos no conexos se les puede determinar su coloramiento solo usando el subgrafo conexo más grande.

3. Algoritmos

3.1. Fuerza bruta

Los algoritmos de Fuerza bruta consisten en evaluar de manera sistemática y exhaustiva todas las opciones posibles hasta encontrar la solución óptima del problema, para poder implementar esta clase de algoritmo al problema del coloramiento se deben evaluar todas las combinaciones de colores posibles en el grafo por cada k -coloramiento posible con una cota superior n (nodos del grafo).

Primeramente la complejidad para validar si un k -coloramiento es posible consta de $O(k^n)$, todas las combinaciones de k colores en los n nodos, además de hacer la revisión de cada mapeo constaría exactamente de $O(n + m)$, donde m es el número de aristas, adicionalmente como se desea encontrar el mínimo coloramiento y no solo si un k -coloramiento es posible se forma de una sumatoria que deba recorrer desde el coloramiento $k = 2$ hasta $k = n$ de la forma:

$$\sum_{k=2}^n k^n = 2^n + \dots + n^n$$

Donde la expresión está dominada por el argumento n^n , con lo cual podemos concluir finalmente que el algoritmo consta de una complejidad temporal de $O(n^n(n + m))$ siendo superexponencial.

Por otro lado la complejidad espacial del algoritmo constaria del mapeo dado, compuesto por la llave(nodo) y el valor(color asignado), para cada combinacin posible que entonces estaria determinado por $O(n)$.

3.2. Backtracking con poda

El backtracking por poda es un técnica algorítmica que construye las soluciones paso por paso, sin embargo poda las ramas del arbol que no cumplen restricciones ya preestablecidas o que ya no son optimas, el algoritmo en este problema recreara todas las combinaciones posibles permitidas, y además al encontrar soluciones de alguna coloración k se descartara cualquier combinación tal que el número de colores sea mayor a k así cortando las ramas donde no se encuentra el mínimo temporal, teniendo en la practica una mejor velocidad contra la metodología de fuerza bruta.

Sin embargo para encontrar la complejidad temporal debemos analizar el peor de los casos, donde no se logran descartar cambios con la poda, con esto en cuenta el peor de los casos estaria acotado de forma factorial ($O(n!)$) y adicionalmente se multiplicara por el costo de validacion de cada estado $O(n + m)$, finalmente obteniendo una complejidad final de $O(n!(n + m))$.

En otro orden de ideas la complejidad espacial estara fijada por las variables que se lleven en la pila de recursion, siendo esta de tamaño n y como todas las varibles usadas son de tamaño constante para la iteración dada la complejidad espacial es definida por $O(n)$

3.3. Voraz

Un algoritmo Voraz consiste en buscar los optimos locales según una heurística específica sin considerar su influencia en la optimilidad global, por esto son considerados como algoritmos miopes, en el problema de coloramiento de grafos se emplea el algoritmo Voraz DSATUR [1], en el cual su heurística consta de priorisar el coloramiento de los vértices que presenten un mayor grado de saturación, el número de sus vecinos que ya han sido coloreados, en casos de empate se considera al nodo de mayor grado, número de vecinos.

El algoritmo busca al nodo de máxima saturación entre los nodos restantes para cada iteración, una cantidad n de veces, por lo que se obtiene una complejidad de $O(n)$, además se debe hacer la actualización de la saturación, trabajo que se basa en pasar por cada arista del grafo generando asi la complejidad de $O(m)$ para esta tarea y condicionando a que la complejidad temporal total sea:

$$O(n^2 + m) \approx O(n^2)$$

Complementado la complejidad espacial esta dada por el conjunto de vértices tamaño n , los diccionarios de los vértices con su color, el vértice con el color de todos su vecinos, su saturación y grado respectivo claramente todos de tamaño n ; y un conjunto temporal de colores usados que su máximo tamaño estara acotado por n , concluyendo que su complejidad espacial es de $O(n)$

3.4. Probabilístico

Los algoritmos de tipo probabilísticos consisten en aleatorizar los métodos con los que se soluciona el problema a tratar, existen dos categorías para estos en primer lugar los que varían su tiempo de ejecución pero siempre llegan a la solución óptima del problema y por otro lado están los que afectan a la optimidad de la solución por tener un tiempo de resolución estable, para el presente estudio utilizaremos la segunda forma, basándonos en el anterior tipo de método podemos notar que un factor relevante para el algoritmo es el orden del coloramiento del grafo, usando la saturación en el caso anterior, ahora usaremos arreglos aleatorizados y ejecutarlos una M cantidad de veces por caso para ver varias soluciones y encontrar la mínima por la instancia.

Su metodología consiste en primero randomizar la lista de nodos, posterior a esto se recorre la lista nodo por nodo y se verifican todas sus aristas para poder hacer la escogencia del color más pequeño permitido, con esto en cuenta debemos recorrer todos los nodos $O(n)$ y todas las aristas dos veces al ser un grafo no dirigido $O(m)$, obtenemos así el algoritmo con menor complejidad espacial $O(n + m)$.

Por último su complejidad espacial constaría de la lista randomizada de vértices y del diccionario de colores, los cuales son claramente de tamaño n , obteniendo de este modo una complejidad espacial lineal ($O(n)$)

4. Diseño Experimental

Como ya fue mencionado en la sección 2, omitiremos algunas instancias consideradas triviales o que no aportan resultados a nuestro estudio.

Para evaluar la corrección y el comportamiento de los algoritmos de coloración, se diseñaron dos grupos de pruebas: pruebas sobre grafos construidos manualmente y pruebas sobre grafos generados aleatoriamente.

Los casos manuales permiten comprobar el funcionamiento de los algoritmos sobre estructuras clásicas cuyo número cromático es conocido previamente. Por otra parte, los grafos aleatorios permiten evaluar los algoritmos sobre instancias con diferentes tamaños y densidades, manteniendo el conocimiento exacto de su número cromático.

4.1. Grafos de prueba construidos manualmente

En total, se incluyeron catorce grafos representativos, seleccionados para evaluar diferentes propiedades estructurales relacionadas con el problema de coloración.

camino_p5 ($\chi(G) = 2$): Camino simple compuesto por cinco vértices. Al ser un árbol con más de un vértice, puede colorearse utilizando exactamente dos colores.

ciclo_par_c4 ($\chi(G) = 2$): Ciclo compuesto por cuatro vértices. Debido a que su longitud es par, el grafo es bipartito y puede colorearse con dos colores.

ciclo_impar_c5 ($\chi(G) = 3$): Ciclo compuesto por cinco vértices. Los ciclos de longitud impar no son bipartitos y requieren exactamente tres colores.

estrella_k1.5 ($\chi(G) = 2$): Grafo estrella formado por un vértice central conectado a cinco vértices periféricos. Esta estructura es bipartita, por lo que requiere dos colores.

bipartito_completo_k2.3 ($\chi(G) = 2$): Grafo bipartito completo $K_{2,3}$, constituido por dos conjuntos independientes de tamaños dos y tres. Cada vértice de un conjunto se encuentra conectado con todos los vértices del otro conjunto.

grafo_casa ($\chi(G) = 3$): Grafo formado por un ciclo de cuatro vértices y un vértice adicional que forma un triángulo sobre uno de sus lados. La presencia de este triángulo hace necesario utilizar tres colores.

grafo_diamante ($\chi(G) = 3$): Grafo obtenido al eliminar una arista del grafo completo K_4 . Aunque deja de ser completo, conserva triángulos que obligan a utilizar al menos tres colores.

dos_triangulos_comparten_vertice ($\chi(G) = 3$): Grafo formado por dos copias de K_3 que comparten un único vértice. Cada triángulo requiere tres colores, por lo que el número cromático del grafo completo es tres.

rueda_sobre_c4 ($\chi(G) = 3$): Grafo rueda formado por un ciclo C_4 y un vértice central conectado con todos los vértices del ciclo. El ciclo exterior utiliza dos colores y el vértice central requiere un tercer color.

rueda_sobre_c5 ($\chi(G) = 4$): Grafo rueda formado por un ciclo C_5 y un vértice central conectado con todos los vértices del ciclo. El ciclo impar requiere tres colores y el vértice central, al ser adyacente a todos ellos, requiere un cuarto color.

tripartito_completo_k2.2.2 ($\chi(G) = 3$): Grafo tripartito completo $K_{2,2,2}$, compuesto por tres conjuntos independientes de dos vértices cada uno. Los vértices pertenecientes a conjuntos diferentes están completamente conectados, por lo que cada conjunto debe utilizar un color distinto.

cuatripartito_completo_k2.2.2.2 ($\chi(G) = 4$): Grafo cuatripartito completo $K_{2,2,2,2}$, compuesto por cuatro conjuntos independientes. Debido a que todos los vértices de conjuntos diferentes son adyacentes, se requieren cuatro colores.

grafo_petersen ($\chi(G) = 3$): Grafo de Petersen, compuesto por diez vértices y quince aristas. Se trata de un grafo regular, no bipartito y libre de triángulos, cuyo número cromático es tres.

crown_graph_4 ($\chi(G) = 2$): Grafo corona de ocho vértices, obtenido a partir del grafo bipartito completo $K_{4,4}$ mediante la eliminación de un emparejamiento perfecto. El grafo continúa siendo bipartito y, por tanto, puede colorearse con dos colores.

La selección de estos grafos permite evaluar casos bipartitos, ciclos pares e impares, grafos con triángulos, grafos multipartitos, estructuras regulares y casos potencialmente adversos para algoritmos heurísticos, como el grafo corona.

4.2. Generación de grafos aleatorios

Un aspecto importante de esta generación consiste en conocer previamente el número cromático de cada instancia.

Generar un grafo completamente aleatorio y determinar posteriormente su número cromático exacto implicaría resolver nuevamente un problema computacionalmente difícil. En particular, calcular el número cromático de un grafo general es un problema NP-difícil.

Para evitar esta dificultad, se utilizó un procedimiento constructivo que genera grafos con una coloración plantada de exactamente k colores. La construcción garantiza matemáticamente que el número cromático del grafo generado es:

$$\chi(G) = k.$$

El procedimiento se divide en las siguientes etapas.

4.2.1. Partición de los vértices en conjuntos independientes

Dados n vértices y un número de colores k , los vértices se desordenan aleatoriamente y se distribuyen entre k grupos no vacíos.

Cada grupo representa una clase de color. Todos los vértices pertenecientes al mismo grupo reciben inicialmente el mismo color, identificado mediante un valor entre 0 y $k - 1$.

Durante la construcción no se agregan aristas entre vértices que pertenezcan al mismo grupo. Por tanto, cada grupo constituye un conjunto independiente.

Esta partición produce una coloración válida con k colores, lo que permite establecer que:

$$\chi(G) \leq k.$$

Es decir, el grafo generado puede colorearse utilizando como máximo k colores.

4.2.2. Inserción de un clique K_k

A continuación, se selecciona un vértice representante de cada uno de los k grupos. Los representantes se conectan entre sí, formando un subgrafo completo K_k .

En un grafo completo K_k , todos los pares de vértices son adyacentes. En consecuencia, cada uno de sus vértices debe recibir un color diferente. Esto implica que cualquier coloración válida del grafo generado debe utilizar al menos k colores:

$$\chi(G) \geq k.$$

Al combinar las dos desigualdades obtenidas:

$$\chi(G) \leq k$$

y

$$\chi(G) \geq k,$$

se concluye que:

$$\chi(G) = k.$$

De esta manera, el número cromático de cada grafo generado es conocido sin necesidad de ejecutar previamente un algoritmo exacto de coloración.

4.2.3. Garantía de conectividad

Después de construir el clique K_k , cada vértice que no haya sido seleccionado como representante se conecta con un representante perteneciente a un grupo diferente.

Esta conexión conserva la validez de la coloración plantada, dado que solo se crean aristas entre vértices que tienen colores diferentes.

Además, debido a que todos los representantes forman parte del clique y cada vértice restante se conecta con al menos uno de ellos, se garantiza que todos los vértices pertenezcan a una única componente conexa. Por tanto, el grafo generado no contiene vértices aislados ni componentes desconectadas.

4.2.4. Adición de aristas aleatorias

Finalmente, se recorren los pares de vértices pertenecientes a grupos diferentes. Para cada par se agrega una arista con una probabilidad determinada, denominada probabilidad de conexión.

Por ejemplo, para una probabilidad de 0,35, cada par válido de vértices tiene una probabilidad del 35 % de ser conectado.

Esta etapa permite generar grafos con diferentes densidades y estructuras, aumentando la diversidad y complejidad de las instancias de prueba. Sin embargo, las aristas únicamente se agregan entre vértices pertenecientes a clases de color diferentes, por lo que la coloración plantada continúa siendo válida.

En consecuencia, la adición de aristas aleatorias no modifica la garantía de que:

$$\chi(G) = k.$$

4.3. Validación de la entrada

Antes de ejecutar los algoritmos, se verifica que cada grafo cumpla con las restricciones establecidas para el proyecto. Primero que todo el grafo debe estar representado mediante un diccionario de listas de adyacencia.

Y algunas funciones de validación comprueban los siguiente:

- La entrada debe ser un diccionario.
- Cada vértice debe estar asociado con una lista de vecinos.
- Todos los vecinos deben existir como vértices dentro del diccionario.
- El grafo debe ser no dirigido.
- No deben existir lazos.
- No deben existir aristas repetidas.
- El grafo debe contener al menos una arista.
- El grafo no debe ser completo.

Para comprobar que el grafo sea no dirigido, se verifica que para toda arista (u, v) también exista la relación inversa (v, u) en las listas de adyacencia.

4.4. Validación de la salida

La validación semántica de los resultados se implementó en el archivo `validators.py`. Su función consiste en verificar que la salida entregada por cada algoritmo cumpla tanto con la estructura definida como con las condiciones propias de una coloración válida.

El primer elemento corresponde al número de colores utilizados y el segundo contiene la asignación de color para cada vértice.

Las validaciones realizadas sobre esta salida incluyen:

Esta validación permite detectar soluciones con colisiones de color, incluso cuando el valor de k reportado por el algoritmo parece correcto.

4.5. Diferenciación entre algoritmos exactos y heurísticos

Las pruebas fueron diseñadas considerando que no todos los algoritmos implementados poseen las mismas garantías.

Los algoritmos exactos, como fuerza bruta y backtracking con poda, deben encontrar el número cromático óptimo del grafo. Por tanto, además de entregar una coloración válida, deben cumplir:

$$k_{obtenido} = \chi(G).$$

En estos casos, la prueba compara directamente el valor retornado por el algoritmo con el número cromático conocido del grafo.

Por otra parte, los algoritmos heurísticos, como greedy, y los métodos probabilísticos no garantizan encontrar siempre la solución óptima. Estos algoritmos pueden producir una coloración válida utilizando una cantidad de colores superior al mínimo.

Por esta razón, para los algoritmos heurísticos se exige que:

$$k_{obtenido} \geq \chi(G),$$

y que la asignación de colores sea completamente válida.

Esta diferenciación evita considerar como error una solución subóptima producida por un algoritmo que, por definición, no garantiza optimalidad. Sin embargo, se mantiene como requisito obligatorio que no existan conflictos de color y que todos los vértices sean coloreados.

En el caso de los algoritmos probabilísticos, las pruebas pueden realizar múltiples ejecuciones utilizando semillas diferentes. Esto permite analizar su estabilidad, registrar el mejor resultado obtenido y determinar con qué frecuencia alcanzan el número cromático óptimo.

5. Implementación

La infraestructura fue diseñada para evaluar tanto la corrección de las funciones auxiliares como el comportamiento de los diferentes algoritmos de coloración. Para ello, las pruebas se dividieron en tres grupos principales: pruebas unitarias para el verificador de coloraciones, pruebas sobre grafos construidos manualmente y pruebas sobre grafos generados aleatoriamente.

5.1. Pruebas parametrizadas

Las pruebas sobre los grafos manuales y aleatorios se implementaron mediante un decorador. Esta funcionalidad permite ejecutar una misma función de prueba utilizando diferentes grafos y diferentes algoritmos.

De esta manera, no fue necesario escribir una prueba independiente para cada combinación de algoritmo y grafo. En su lugar, cada caso se almacena junto con su nombre, su representación mediante listas de adyacencia y su número cromático esperado.

Cada elemento de la lista de casos es enviado automáticamente a la función de prueba. Esto permite ampliar la cobertura agregando nuevos grafos a la colección sin modificar la lógica principal de las pruebas.

Asimismo, los algoritmos pueden registrarse en una lista de configuración, indicando si corresponden a métodos exactos o heurísticos. La prueba recibe tanto el algoritmo como el grafo y aplica las validaciones correspondientes a cada tipo de método.

5.2. Resultado de la implementación

La estructura implementada permite evaluar los algoritmos mediante una infraestructura común y reutilizable. Cada algoritmo recibe grafos con la misma representación y debe producir resultados con el mismo formato de salida.

La combinación de grafos manuales, grafos aleatorios con número cromático conocido y validaciones semánticas permite comprobar tres aspectos principales:

- Que los algoritmos respeten el formato definido para la entrada y la salida.
- Que las asignaciones de colores producidas sean válidas.
- Que los algoritmos exactos encuentren la solución óptima y que los heurísticos produzcan soluciones válidas, aunque puedan ser subóptimas.

Esta implementación facilita la comparación entre los diferentes métodos y permite ampliar posteriormente el conjunto de pruebas sin modificar la estructura general del sistema.

6. Resultados

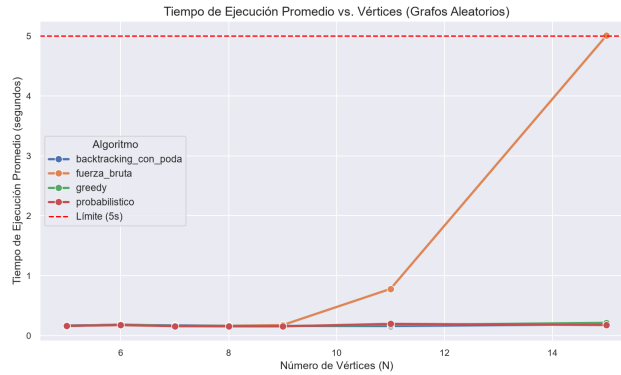


Figura 1: Crecimiento del tiempo de ejecución de cada algoritmo por vértices.

Una gráfica que nos representa el tiempo de ejecución, además presenta un límite temporal determinante para conocer que tipo de algoritmo al sobre pasar esta cota se vuelve ineficiente desde una cantidad pequeña de vértices para los grafos dados.

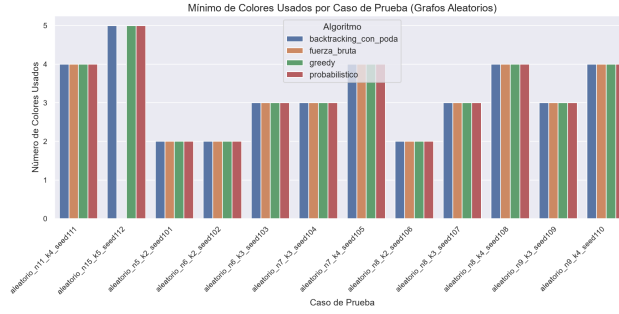


Figura 2: Colores encontrados por cada algoritmo

Una gráfica de barras la cual nos muestra el coloramiento óptimo encontrado por cada algoritmo para once grafos distintos con distintos coloramientos predefinidos, se tiene en cuenta que la solución $\chi(G)$ es encontrada por los algoritmos deterministas de Bactracking y Fuerza bruta.

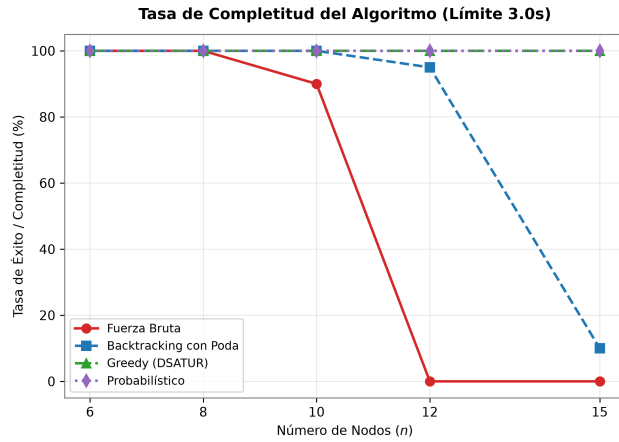


Figura 3: Completitud de las ejecuciones de los algoritmos

Gráfica de completitud, nos demuestra cual es el porcentaje de instancias que son solucionables en menos de una cota temporal (tres segundos) para cada algoritmo, los cual persive la factibilidad de los algoritmos para resolver grafos de diferentes tamaños en un rango temporal.

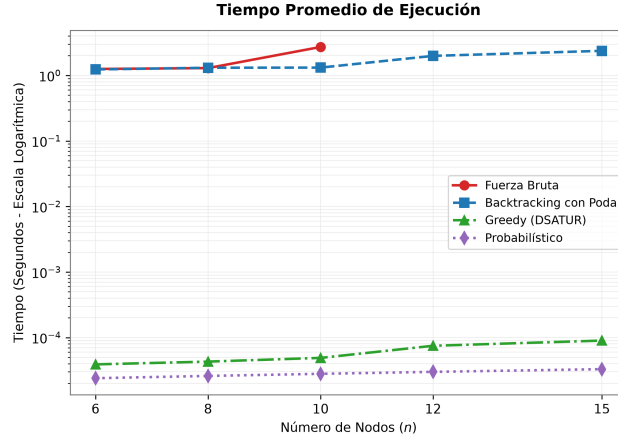


Figura 4: Tiempo de ejecución en escala logarítmica
Representación logarítmica donde se evidencia el comportamiento temporal de los algoritmos y como conservan un orden en el tiempo de ejecución con respecto al número de vértices.

7. Discusión

Los resultados obtenidos evidencian diferencias importantes entre los algoritmos en términos de escalabilidad, aun cuando todos produjeron soluciones con la misma cantidad de colores en las instancias evaluadas.

Como se observa en la Figura 1, el tiempo de ejecución de fuerza bruta aumenta considerablemente al incrementar el número de vértices. El crecimiento se vuelve especialmente pronunciado en las instancias de mayor tamaño, hasta alcanzar el límite de tiempo establecido. En contraste, backtracking con poda presenta un crecimiento más controlado, lo que indica que la eliminación anticipada de soluciones inviables reduce de manera considerable el espacio de búsqueda explorado.

La Figura 2 muestra que los cuatro algoritmos encontraron la misma cantidad de colores para cada uno de los grafos aleatorios utilizados. Se debe aclarar, que en la segunda instancia el algoritmo de fuerza bruta no encontró la solución óptima en el tiempo establecido. Por otra parte, en estas instancias, los métodos greedy y probabilístico lograron igualar las soluciones obtenidas por los algoritmos exactos. Sin embargo, este resultado debe interpretarse únicamente dentro del conjunto de pruebas realizado, ya que no constituye una garantía de que los métodos heurísticos encuentren siempre el número cromático óptimo en grafos con otras estructuras.

La tasa de completitud presentada en la Figura 3 permite observar con mayor claridad las limitaciones de los métodos exactos. Fuerza bruta comienza a presentar ejecuciones incompletas desde los diez vértices y deja de finalizar dentro del límite establecido para los grafos de doce y quince vértices. Backtracking

con poda mantiene una tasa de completitud elevada hasta los doce vértices, aunque también presenta una disminución considerable al llegar a quince. Por su parte, greedy y el algoritmo probabilístico completaron la totalidad de las ejecuciones.

La representación en escala logarítmica de la Figura 4 permite comparar algoritmos cuyos tiempos se encuentran en órdenes de magnitud diferentes. Los métodos heurísticos presentan tiempos cercanos a las diezmilésimas de segundo, mientras que los algoritmos exactos requieren tiempos del orden de segundos. Esta diferencia muestra que la búsqueda de una solución óptima tiene un costo computacional considerablemente mayor que la obtención rápida de una coloración válida.

En conjunto, los resultados muestran un compromiso entre exactitud y eficiencia. Fuerza bruta resulta útil únicamente para instancias pequeñas, mientras que backtracking con poda permite ampliar el tamaño de los grafos que pueden resolverse exactamente. Greedy y el método probabilístico ofrecen una mayor capacidad de escalamiento y, para los casos estudiados, alcanzaron soluciones equivalentes a las exactas con tiempos de ejecución considerablemente menores.

8. Conclusiones

Gracias a los resultados y a la discusión anteriormente presentada, podemos concluir que:

- Todos los algoritmos evaluados llegaron a soluciones óptimas para la coloración de grafos en distintas instancias. Sin embargo, esto no quiere decir que los algoritmos greedy y probabilístico siempre lleguen a la solución óptima en cualquier otro tipo de instancia.
- Es fácilmente comprobable, tanto en la teoría como en la práctica, que el algoritmo de fuerza bruta presenta un crecimiento exponencial en el tiempo de ejecución, a pesar de encontrar la mejor solución para los problemas. Por esto, su mejor uso es en instancias pequeñas que no requieran un alto costo computacional.
- El backtracking con poda presentó una gran mejora frente a fuerza bruta, ya que la poda permite evitar caminos o combinaciones que no son óptimas para el problema. No obstante, su tiempo de ejecución también es alto, al igual que el de fuerza bruta, por lo que es recomendable para instancias pequeñas.
- Los algoritmos greedy y probabilístico presentaron un tiempo de ejecución bajo para el problema y llegaron a una solución óptima para el conjunto de pruebas realizadas. Sin embargo, se debe tener en cuenta que estos resultados no se pueden generalizar a todo tipo de grafo.
- Los resultados evidencian un compromiso entre optimalidad y eficiencia. Los algoritmos exactos permiten conocer la mejor solución, pero sacrifican el costo computacional y el tiempo de ejecución. Por otra parte, los

heurísticos permiten obtener bajos tiempos de ejecución, pero no garantizan encontrar siempre la mejor solución.

- La estructura del grafo influye directamente en la dificultad de la coloración. Por tanto, el número de vértices no debe considerarse como el único indicador de complejidad, ya que factores como la densidad, el orden de procesamiento, la presencia de cliques y la distribución de las aristas pueden modificar considerablemente el comportamiento de los algoritmos.
- La generación constructiva de grafos aleatorios permitió conocer previamente su número cromático exacto. Esto hizo posible comparar de forma objetiva los resultados de los algoritmos sin necesidad de calcular posteriormente la solución óptima mediante un procedimiento adicional.

En conclusión, fuerza bruta es útil como referencia en instancias pequeñas, mientras que backtracking con poda constituye una alternativa más eficiente cuando se requiere una solución exacta. Para grafos de mayor tamaño, greedy y el algoritmo probabilístico ofrecen mejores tiempos de ejecución y una alta capacidad de completitud, aunque sus resultados deben interpretarse como soluciones aproximadas cuando no se dispone de una garantía de optimalidad.

Referencias

- [1] Sergio Pena Seijas. El problema de coloración de grafos. Trabajo de fin de máster, Universidade de Santiago de Compostela, 2017. Dirigido por Silvia Lorenzo Freire.
- [2] Javier Ramírez Rodríguez. *Extensiones del Problema de Coloración de Grafos*. Tesis doctoral, Universidad Complutense de Madrid, 2001. ISBN: 84-669-1855-8. Dirigido por Francisco Javier Yáñez Gestoso.